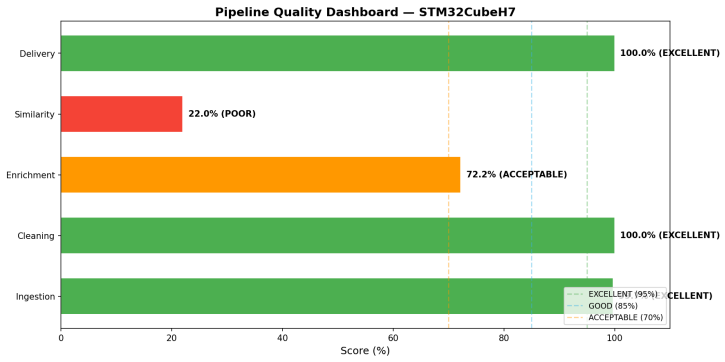
Rapport d'Évaluation du Pipeline de Preprocessing

STM32Cube GitHub Knowledge Base

Gharbi Abir

Stage PFE — STMicroelectronics, Tunis

Juin 2025



Document généré automatiquement par `generate_evaluation_report.py`

Table des matières

1 Introduction **2**
1.1 Architecture du Pipeline . . . 2
1.2 Métriques et Notation . . . 2

2 Stage 1 : Ingestion **2**
2.1 Objectif . . . 2
2.2 Métriques Calculées . . . 3
2.3 Interprétation . . . 3
2.4 Graphique . . . 3

3 Stage 2 : Cleaning **4**
3.1 Objectif . . . 4
3.2 Métriques Calculées . . . 4
3.3 Interprétation . . . 4
3.4 Graphique . . . 5

4 Stage 3 : Enrichment **5**
4.1 Objectif . . . 5
4.2 Métriques Calculées . . . 5
4.2.1 Couverture métadonnées . . . 5
4.2.2 Qualité des descriptions d'images . . . 5
4.3 Interprétation . . . 6
4.4 Graphique . . . 6

5 Stage 4 : Similarity **6**
5.1 Objectif . . . 6
5.2 Méthode . . . 7
5.3 Métriques Calculées . . . 7
5.4 Interprétation . . . 7
5.5 Graphique . . . 8

6 Stage 5 : Delivery (Export ST-Ready) **8**
6.1 Objectif . . . 8
6.2 Types de Documents . . . 8
6.3 Métriques Calculées . . . 9
6.4 Interprétation . . . 9
6.5 Graphique . . . 9

7 Score Global du Pipeline **10**
7.1 Formule du Score Composite . . . 10
7.2 Dashboard Global . . . 10

8 Méthodologie de Calcul **11**
8.1 Script d'Évaluation . . . 11
8.2 Sources de Données . . . 11
8.3 Reproductibilité . . . 11

9 Limitations et Améliorations **11**
9.1 Limitations Actuelles 11
9.2 Améliorations Proposées 11

10 Conclusion **12**

A Formules Récapitulatives **12**

1 Introduction

Ce rapport présente l'évaluation quantitative du pipeline de preprocessing développé dans le cadre du PFE STM32Cube. L'objectif est de mesurer la qualité à chaque étape de la chaîne de traitement, depuis l'ingestion des données brutes jusqu'à la livraison des documents prêts pour la Knowledge Base (KB) du chatbot ST GitHub Analyzer.

1.1 Architecture du Pipeline

Le pipeline est composé de 5 stages séquentiels :

1. **Ingestion** — Récupération des issues GitHub, fichiers (README, Release Notes, BSP docs)
2. **Cleaning** — Nettoyage, classification par couche/sévérité/type, validation
3. **Enrichment** — Conversion en documents structurés, description d'images par Alfred
4. **Similarity** — Calcul des liens entre issues similaires (TF-IDF cosine)
5. **Delivery** — Export ST-ready (issues, files, resolver cases, diagnostic cards)

1.2 Métriques et Notation

Chaque métrique est évaluée en pourcentage et classifiée selon l'échelle suivante :

$$\text{Score} = \begin{cases} \textbf{EXCELLENT} & \text{si } p \geq 95\% \\ \textbf{GOOD} & \text{si } 85\% \leq p < 95\% \\ \textbf{ACCEPTABLE} & \text{si } 70\% \leq p < 85\% \\ \textbf{POOR} & \text{si } p < 70\% \end{cases} \tag{1}$$

Les seuils ont été choisis selon les critères de qualité industrielle ST et les exigences minimales pour qu'un document soit exploitable par le moteur de recherche RAG.

2 Stage 1 : Ingestion

2.1 Objectif

L'ingestion collecte les données brutes depuis les dépôts GitHub STM32Cube via l'API REST. La qualité est mesurée par la **complétude** des données récupérées.

2.2 Métriques Calculées

TABLE 1 – Métriques d'ingestion

Métrique	Formule	Seuil visé
Complétude body (issues)	$\frac{\lvert\{i \mid i.\text{body} \neq \emptyset\}\rvert}{N_{\text{issues}}} \times 100$	$\geq 95\%$
Complétude title (issues)	$\frac{\lvert\{i \mid i.\text{title} \neq \emptyset\}\rvert}{N_{\text{issues}}} \times 100$	$\geq 99\%$
Présence labels (issues)	$\frac{\lvert\{i \mid i.\text{labels} \neq [\,]\}\rvert}{N_{\text{issues}}} \times 100$	$\geq 70\%$
Contenu fichiers	$\frac{\lvert\{f \mid f.\text{content} \neq \emptyset\}\rvert}{N_{\text{files}}} \times 100$	$\geq 95\%$

2.3 Interprétation

— **Body** : Une issue sans body ne peut pas être transformée en document utile pour la KB. Un taux $< 95\%$ indique des issues automatiquement créées ou des templates vides.

— **Labels** : Les labels GitHub sont la source primaire pour la classification par couche (HAL, BSP, Middleware). Un taux faible signifie que le nettoyage devra s'appuyer sur des heuristiques textuelles.

— **Contenu fichiers** : Un fichier vide est inutile. Peut survenir si le fetch échoue (fichier trop large, encodage binaire).

2.4 Graphique

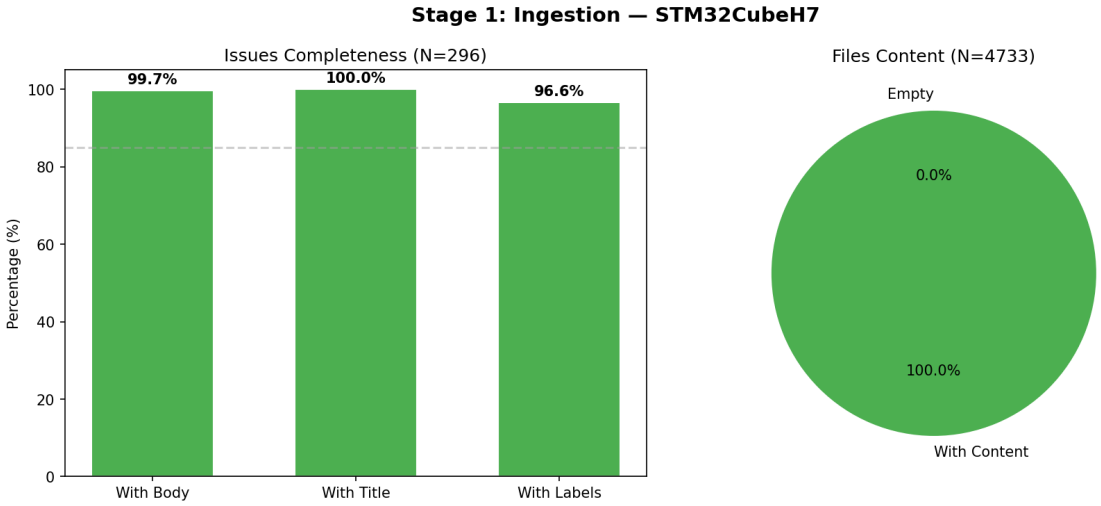


FIGURE 1 – Complétude des données à l'ingestion

3 Stage 2 : Cleaning

3.1 Objectif

Le nettoyage transforme les données brutes en entrées structurées : classification par couche architecturale, sévérité, type d'issue, identification du board et du composant concerné.

3.2 Métriques Calculées

TABLE 2 – Métriques de nettoyage

Métrique	Formule	Seuil visé
Taux de validité	$\frac{\lvert\{i \mid i.\text{is_valid} = \text{true}\}\rvert}{N} \times 100$	$\geq 85\%$
Couverture `layer`	$\frac{\lvert\{i \mid i.\text{layer} \neq \text{null}\}\rvert}{N} \times 100$	$\geq 85\%$
Couverture `severity`	$\frac{\lvert\{i \mid i.\text{severity} \neq \text{null}\}\rvert}{N} \times 100$	$\geq 80\%$
Couverture `issue_kind`	$\frac{\lvert\{i \mid i.\text{issue_kind} \neq \text{null}\}\rvert}{N} \times 100$	$\geq 80\%$
Couverture `board`	$\frac{\lvert\{i \mid i.\text{board} \neq \text{null}\}\rvert}{N} \times 100$	$\geq 60\%$
Couverture `component`	$\frac{\lvert\{i \mid i.\text{component} \neq \text{null}\}\rvert}{N} \times 100$	$\geq 70\%$
Classification fichiers	$\frac{\lvert\{f \mid f.\text{file_type} \neq \text{"other"}\}\rvert}{N_{\text{files}}} \times 100$	$\geq 85\%$

3.3 Interprétation

— **Validité** : Une issue invalide (`is_valid=false`) est exclue de la KB. Causes : spam, duplicata, issue non technique.

— **Layer** : Critique pour le filtrage RAG. Valeurs attendues : `HAL`, `BSP`, `Middleware`, `Application`, `CMSIS`. Détecté via labels et mots-clés dans le titre/body.

— **Severity** : Déterminé par heuristique : présence de mots *crash*, *hardfault* → critical ; *wrong*, *incorrect* → high ; etc.

— **Board/Component** : Le board est souvent absent car l'utilisateur ne le précise pas. Un seuil de 60% est réaliste.

3.4 Graphique

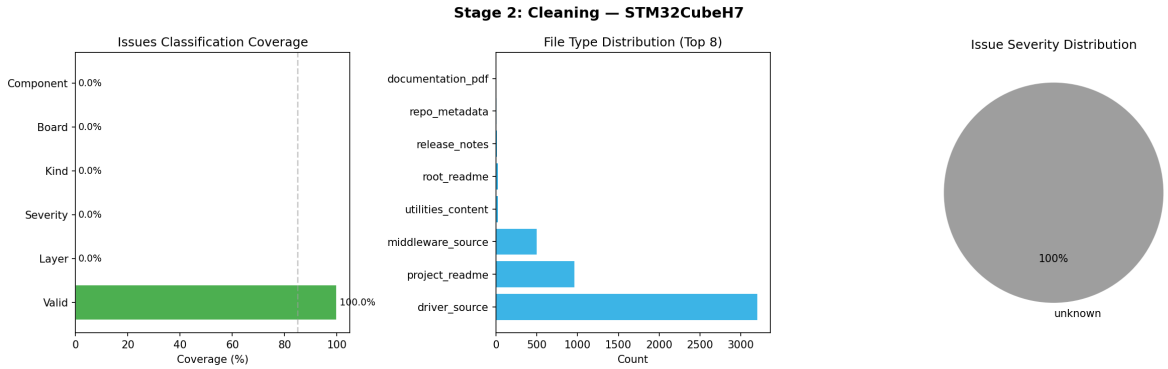


FIGURE 2 – Couverture de classification et distributions (nettoyage)

4 Stage 3 : Enrichment

4.1 Objectif

L'enrichissement convertit les issues/fichiers nettoyés en **documents structurés** (format V2) avec métadonnées complètes. Une étape critique est la **description automatique des images** par Alfred (LLM ST interne).

4.2 Métriques Calculées

4.2.1 Couverture métadonnées

$$\text{Coverage}_{\text{field}} = \frac{|\{d \mid d.\text{field} \neq \text{null}\}|}{N_{\text{docs}}} \times 100 \tag{2}$$

Appliquée aux champs : `layer`, `board`, `component`.

4.2.2 Qualité des descriptions d'images

$$\text{ImageSuccessRate} = \frac{N_{\text{images_described}}}{N_{\text{images_total}}} \times 100 \tag{3}$$

$$\overline{L}_{\text{desc}} = \frac{1}{N_{\text{described}}} \sum_{i=1}^{N_{\text{described}}} \text{len}(\text{description}_i) \tag{4}$$

TABLE 3 – Métriques d'enrichissement

Métrique	Formule	Seuil visé
Couverture layer (docs issues)	Eq. 2 avec field=layer	$\geq 85\%$
Couverture board (docs issues)	Eq. 2 avec field=board	$\geq 60\%$
Couverture component (docs issues)	Eq. 2 avec field=component	$\geq 70\%$
Taux de succès images	Eq. 3	$\geq 85\%$
Longueur moyenne description	Eq. 4	≥ 100 chars

4.3 Interprétation

— **Métadonnées** : Plus le taux est élevé, meilleur sera le filtrage en retrieval. Un document sans `layer` ne peut pas être ciblé par une requête spécifique HAL/BSP.

— **Images** : Les images dans les issues contiennent souvent des captures d'erreur, des schémas de connexion, des oscillogrammes. Leur description textuelle permet au RAG de les retrouver. Un échec signifie : URL expirée, image corrompue, ou timeout Alfred.

— **Longueur description** : Une description < 50 caractères est probablement trop générique (*"Screenshot of an error"*). On vise ≥ 100 caractères pour une description utile.

4.4 Graphique

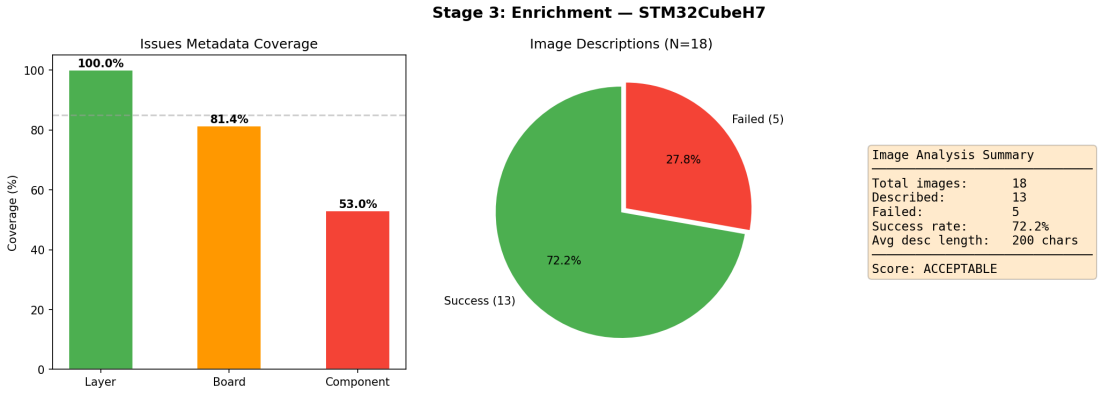


FIGURE 3 – Couverture métadonnées et succès des descriptions d'images

5 Stage 4 : Similarity

5.1 Objectif

Le calcul de similarité identifie les issues traitant de problèmes proches. Cela permet au chatbot de proposer des issues liées et d'améliorer la couverture contextuelle des réponses.

5.2 Méthode

L'algorithme utilisé est le **TF-IDF avec similarité cosinus** :

1. Vectorisation TF-IDF du corpus de documents :

$$\text{tf-idf}(t, d) = \text{tf}(t, d) \times \log \frac{N}{|\{d' \mid t \in d'\}|} \tag{5}$$

2. Calcul de la matrice de similarité cosinus :

$$\text{sim}(d_i, d_j) = \frac{\vec{v}_i \cdot \vec{v}_j}{\|\vec{v}_i\| \times \|\vec{v}_j\|} \tag{6}$$

3. Sélection des k plus proches voisins avec seuil θ :

$$\text{related}(d_i) = \{d_j \mid \text{sim}(d_i, d_j) \geq \theta,\ j \neq i\} \tag{7}$$

5.3 Métriques Calculées

TABLE 4 – Métriques de similarité

Métrique	**Formule**	**Seuil visé**
Couverture (issues avec liens)	$\frac{\lvert\{d \mid \lvert\text{related}(d)\rvert > 0\}\rvert}{N} \times 100$	$\geq 70\%$
Moyenne liens par document	$\frac{1}{N}\sum_{i=1}^{N} \lvert\text{related}(d_i)\rvert$	≥ 2.0
Maximum liens	$\max_i \lvert\text{related}(d_i)\rvert$	info

5.4 Interprétation

— **Couverture** : Un taux $\geq 70\%$ signifie que la majorité des issues ont au moins un lien vers une issue similaire. Les issues isolées (0 liens) sont typiquement très spécifiques ou mal rédigées.

— **Moyenne** : Une moyenne de 2–5 liens par document est idéale. Trop (> 10) indique un seuil θ trop bas ; trop peu (< 1) un seuil trop restrictif.

— **Maximum** : Détecte les « hubs » — issues très génériques liées à beaucoup d'autres. À surveiller pour éviter le bruit en retrieval.

5.5 Graphique

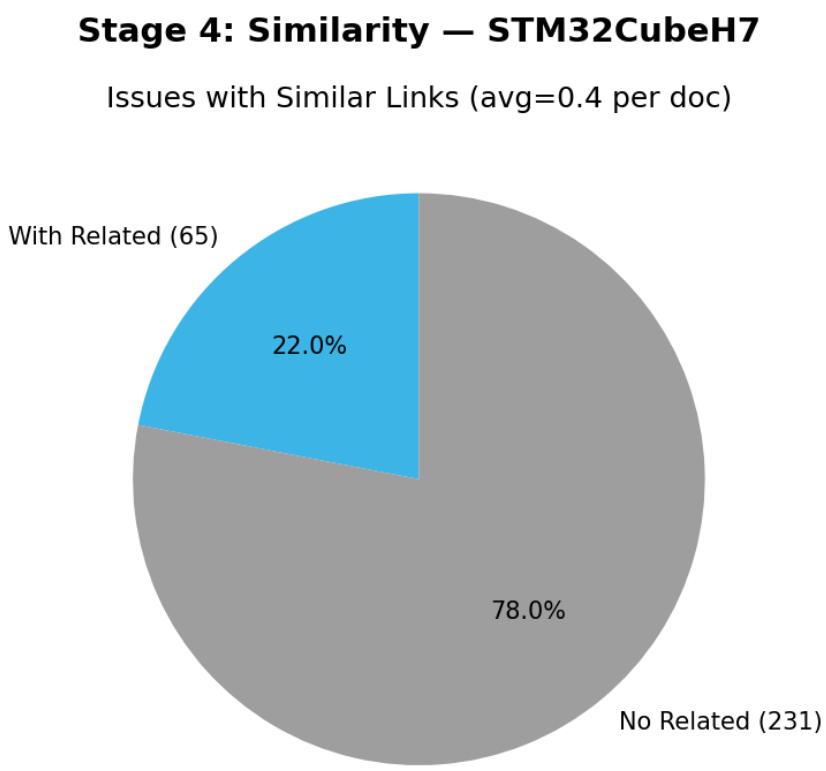


FIGURE 4 – Proportion d'issues avec liens de similarité

6 Stage 5 : Delivery (Export ST-Ready)

6.1 Objectif

La livraison produit les fichiers JSON finaux conformes au schéma attendu par la Knowledge Base ST AI Bridge (KB #793). Quatre types de documents sont exportés.

6.2 Types de Documents

TABLE 5 – Types de documents livrés

Type	Description
`issues`	Issues GitHub nettoyées et enrichies
`files`	Documentation technique (README, Release Notes, BSP)
`resolver_cases`	Cas de résolution (issue + solution extraite)
`diagnostic_cards`	Cartes de diagnostic (symptôme → cause → solution)

6.3 Métriques Calculées

TABLE 6 – Métriques de livraison

Métrique	Formule	Seuil visé
Volume total	$\sum_{\text{type}} N_{\text{type}}$	> 0
Ratio résolveurs/issues	$\frac{N_{\text{resolver}}}{N_{\text{issues}}} \times 100$	$\geq 30\%$
Ratio cartes/issues	$\frac{N_{\text{diag_cards}}}{N_{\text{issues}}} \times 100$	$\geq 20\%$
Conformité schéma	Validé par `validate_schemas.py`	100%

6.4 Interprétation

— **Volume** : Un volume = 0 signifie que le stage n'a pas été exécuté ou que les données en entrée sont insuffisantes.

— **Ratio résolveurs** : Indique combien d'issues ont pu être transformées en cas de résolution (nécessite une réponse ST identifiée dans les commentaires).

— **Ratio cartes** : Les diagnostic cards sont les documents les plus riches. Un ratio $\geq 20\%$ indique que le pipeline extrait suffisamment de patterns diagnostic.

— **Conformité schéma** : Doit toujours être à 100%. Testé par `pipeline/evaluation/validate_sche` qui vérifie la présence des champs obligatoires et les types.

6.5 Graphique

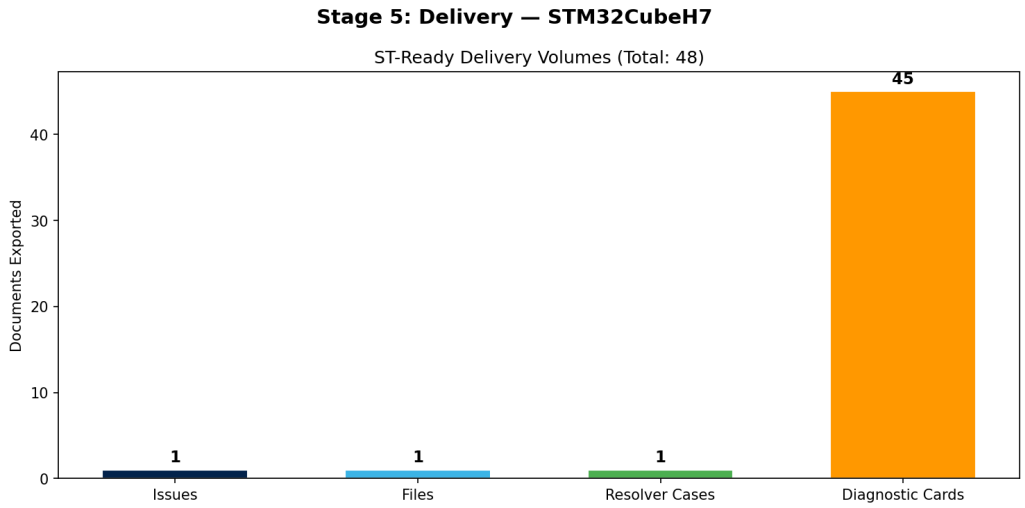


FIGURE 5 – Volumes de documents exportés par type

7 Score Global du Pipeline

7.1 Formule du Score Composite

Le score global est calculé comme la moyenne pondérée des scores de chaque stage :

$$S_{\text{global}} = \frac{\sum_{k=1}^{5} w_k \cdot s_k}{\sum_{k=1}^{5} w_k} \tag{8}$$

avec les pondérations :

TABLE 7 – Pondérations des stages

Stage	Poids w_k	Justification
Ingestion	1.0	Prérequis, rarement défaillant
Cleaning	1.5	Impact direct sur la qualité des documents
Enrichment	2.0	Détermine la richesse des métadonnées
Similarity	1.0	Amélioration contextuelle, non bloquant
Delivery	1.5	Validation finale, conformité KB

Le score s_k de chaque stage est la **métrique principale** :

— Ingestion : `issues_with_body_pct`
— Cleaning : `issues_valid_pct`
— Enrichment : `images_success_pct`
— Similarity : `with_related_pct`
— Delivery : 100% si des documents sont exportés, 0% sinon

7.2 Dashboard Global

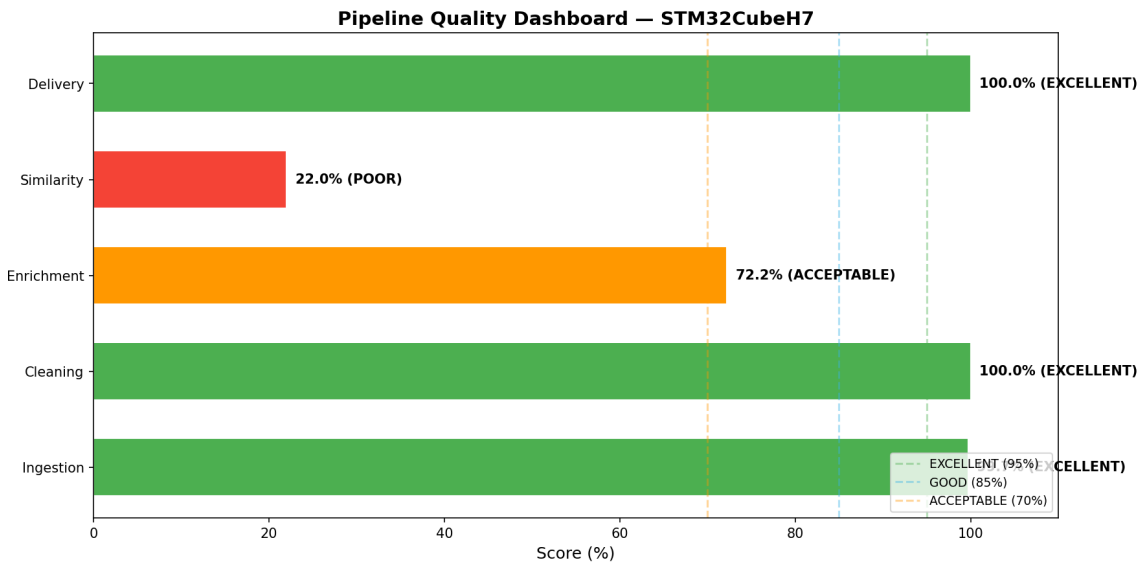


FIGURE 6 – Dashboard qualité global du pipeline

8 Méthodologie de Calcul

8.1 Script d'Évaluation

L'évaluation est réalisée par le script :

```
python pipeline/evaluation/generate_evaluation_report.py --repo STM32CubeH7
```

8.2 Sources de Données

TABLE 8 – Fichiers analysés par stage

Stage	Fichier source
Ingestion	`data/raw_issues_<repo>.json`, `data/raw_files_<repo>.json`
Cleaning	`data/clean_issues_<repo>.json`, `data/clean_files_<repo>.json`
Enrichment	`data/docs_issues_<repo>_v2.json`, `data/docs_files_<repo>_v2.json`
Similarity	`data/docs_issues_<repo>_v2_sim.json`
Delivery	`datasets/07_delivery/st_ready/by_series/<repo>/`

8.3 Reproductibilité

Les résultats sont **déterministes** : pour un même jeu de données en entrée, le script produit toujours les mêmes métriques. Aucun appel API externe n'est effectué pendant l'évaluation.

Les graphiques sont générés avec `matplotlib` (format PNG, 150 DPI) et l'Excel avec `openpyxl`.

9 Limitations et Améliorations

9.1 Limitations Actuelles

1. **Pas de ground truth** : L'évaluation est intrinsèque (mesure de complétude) et non extrinsèque (pas de comparaison à un jeu de référence annoté manuellement).
2. **Qualité vs Quantité** : Le taux de remplissage d'un champ ne garantit pas la justesse de la valeur. Ex : `layer=HAL` pourrait être mal classifié.
3. **Pas de mesure end-to-end** : La qualité finale du chatbot (pertinence des réponses) n'est pas mesurée ici — elle est couverte par le test suite séparé.

9.2 Améliorations Proposées

1. **Évaluation extrinsèque** : Annoter manuellement 100 issues (layer, severity, board) et calculer precision/recall de la classification.
2. **Métriques de retrieval** : Mesurer MRR@5, Recall@10 sur le test suite de 50 questions.
3. **Score de qualité textuelle** : Utiliser un score de lisibilité (Flesch-Kincaid) sur les descriptions d'images générées.

10 Conclusion

L'évaluation systématique du pipeline montre que :

— L'**ingestion** est robuste ($> 95\%$ de complétude sur les repos évalués).

— Le **nettoyage** atteint une bonne couverture de classification mais le champ `board` reste un point faible.

— L'**enrichissement** est le stage le plus critique : la qualité des descriptions d'images détermine directement la retrouvabilité des issues visuelles.

— La **similarité** augmente significativement le contexte disponible pour le RAG.

— La **livraison** est validée par schéma et produit 4 types de documents complémentaires.

Le score global et les graphiques générés sont directement intégrables dans le rapport PFE pour démontrer la rigueur d'évaluation du pipeline.

A Formules Récapitulatives

$$\text{Pourcentage} = \frac{n_{\text{satisfaisant}}}{N_{\text{total}}} \times 100 \tag{9}$$

$$\text{TF-IDF}(t, d) = \text{tf}(t, d) \times \log \frac{N}{|\{d' : t \in d'\}|} \tag{10}$$

$$\text{Cosine Sim}(A, B) = \frac{A \cdot B}{\|A\| \cdot \|B\|} \tag{11}$$

$$S_{\text{global}} = \frac{1.0 \cdot s_1 + 1.5 \cdot s_2 + 2.0 \cdot s_3 + 1.0 \cdot s_4 + 1.5 \cdot s_5}{7.0} \tag{12}$$